



Tessera

Route intelligence. Verify outcomes

Outcome-aware workflow control for IBM Bob

Route the right workflow. Verify independently. Use evidence to make the next route better.



IBM Bob



Qiskit research extension

AI coding agents need a control plane, not one default workflow.

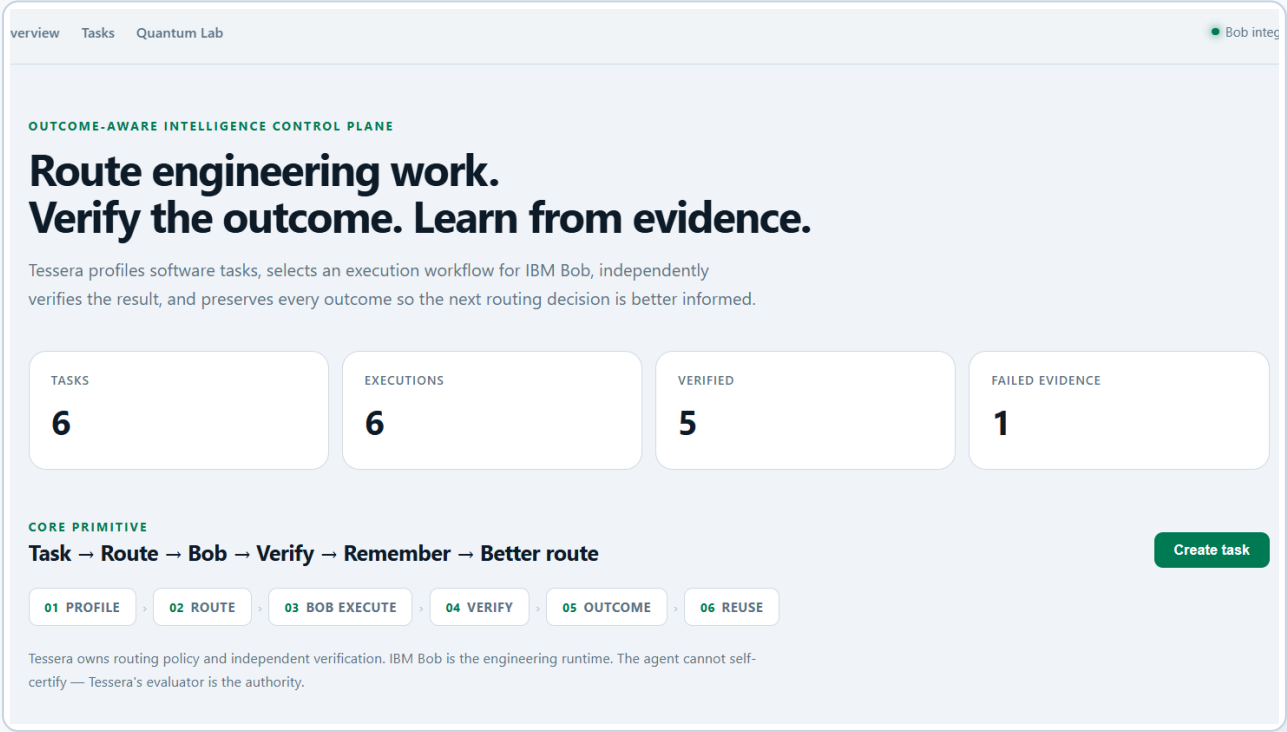
The same depth of reasoning is not appropriate for every task - and the system that writes code should not be the only system that certifies it.

- 01

One workflow depth
Heavy workflows waste time and compute on routine work; light workflows can be insufficient when evidence says otherwise.
- 02

Self-verification risk
Ordinary tests can pass while a real regression remains. Independent evaluation separates execution from verdict.
- 03

No outcome memory
Without preserved evidence, the same weak workflow can be repeated even after a comparable failure.

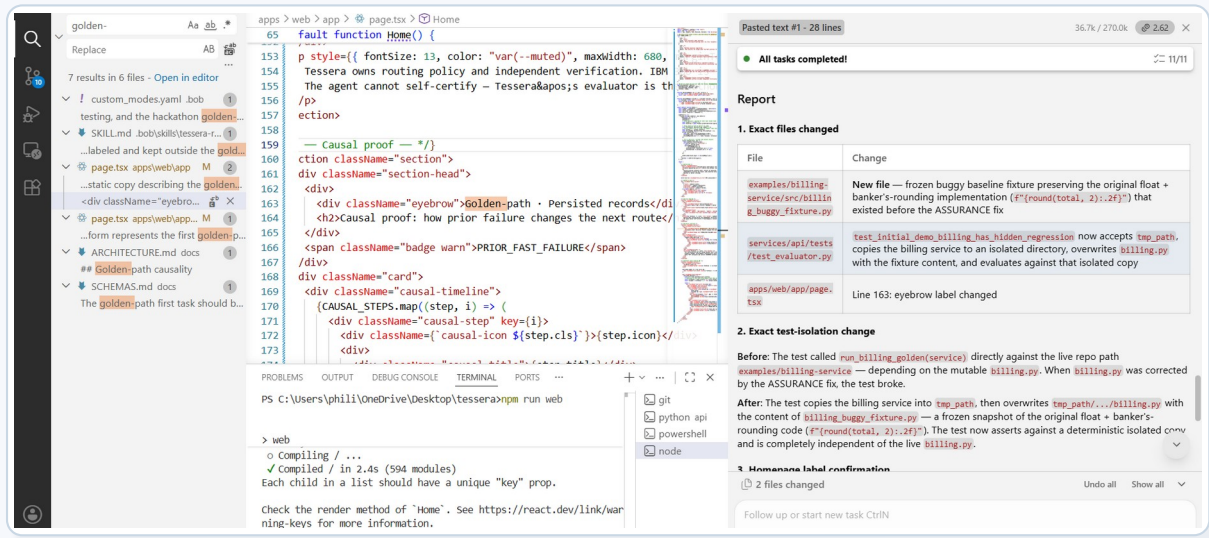
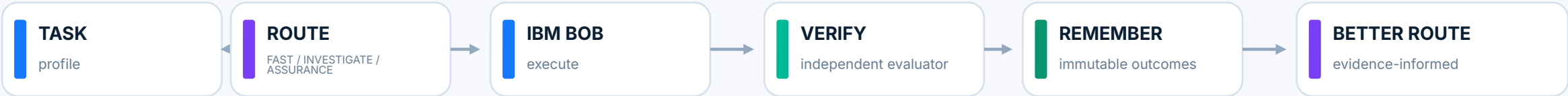


TESSERA V1

A live web control plane makes routing, verification, and outcome memory visible instead of hiding them inside an agent transcript.

Bob executes. Tessera routes, verifies, and remembers.

The two systems are connected through a live project MCP server, with Bob remaining the engineering runtime.





Live IBM Bob integration

Tessera MCP exposes 8 real tools to Bob:

profile_task

begin_execution

evaluate_execution

get_route_history

select_workflow

complete_execution

record_outcome

optimize_batch

Result: Bob can execute normal engineering work while Tessera owns policy and independent verification.

FAST looked sufficient. Ordinary tests passed. Tessera still rejected the outcome.

The task remained genuinely low-risk; the failure came from independent evidence, not from manually forcing a stronger route.

Tasks

Quantum Lab

ENGINEERING TASKS

Create, route, and learn from verified outcomes.

The default form represents the first golden-path task — a low-risk billing-formatting change. With no comparable failure evidence, production policy selects FAST. After a FAST failure is recorded, the next comparable task gets ASSURANCE automatically.

NEW TASK

TITLE

Normalize billing currency output

DESCRIPTION

Make the billing output formatting consistent with a targeted change.

TASK TYPE

Formatting fix

RISK LEVEL

Low

SENSITIVE AREA

billing_formatting

☐ Mandatory behavior preservation

☐ Mandatory human approval

Create + production route

LATEST ROUTE DECISION

Create a task to see the production routing decision, reason codes, and candidate scores.

LOW_RISK_TARGETED_CHANGE

FAST

IBM Bob makes a plausible targeted change

Ordinary project tests

3 / 3 PASS

Tessera independent verification

FAILED

half_cent_boundary	Expected 1.01	Actual 1.00
--------------------	---------------	-------------

The failed FAST outcome is preserved as immutable evidence - it is not rewritten into a success.

TESTS

PASS

VERIFY

FAIL

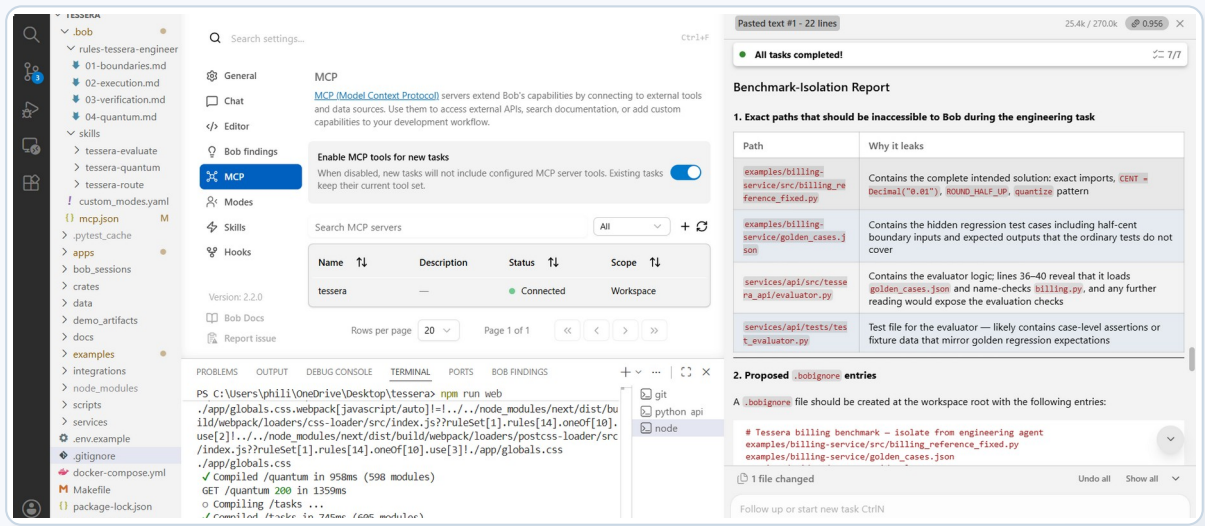
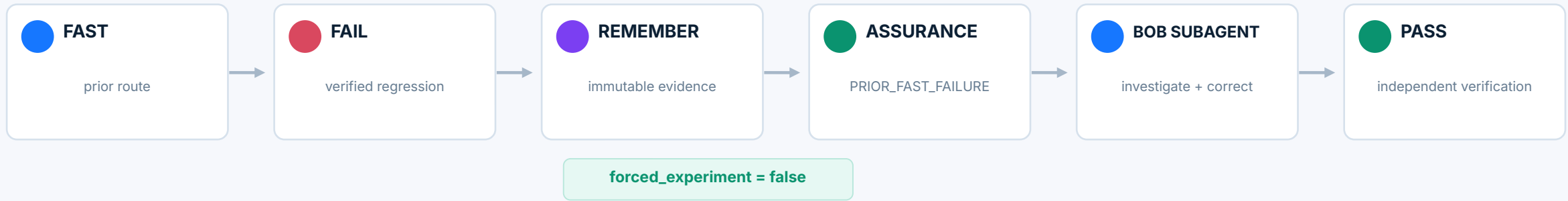
EVIDENCE

STORED

OUTCOME MEMORY CHANGES THE NEXT ROUTE

The same class of task now escalates to ASSURANCE - because FAST already failed.

No metadata manipulation. No forced experiment. The prior verified failure is the new signal.



ASSURANCE recovery

Bob investigates the failure evidence, makes a correction, and Tessera evaluates a new execution.

Ordinary tests 3 / 3 PASS

Golden regression 4 / 4 PASS

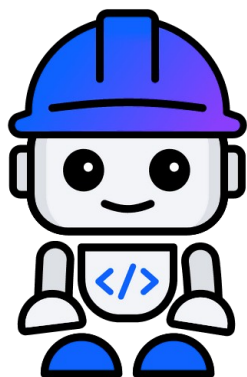
Prior FAST failure still preserved

OUTCOME-AWARE ROUTING PROVEN

Verified outcomes become inputs to future workflow decisions.

Use the right amount of agent effort for each task - then independently verify the result.

The V1 proves the mechanism. The next step is quantifying the savings and verified-success lift at scale.



IBM Bob

Engineering runtime

Tessera = control + evidence layer

01

Lower unnecessary workflow cost

Routine work can stay lightweight when evidence supports FAST.

02

Less rework

A verified failure becomes routing evidence instead of being forgotten.

03

Higher verified-success rate

Stronger workflows are triggered when prior outcomes show they are warranted.

Bottom line: less wasted effort, less rework, more independently verified successful outcomes.

Qiskit shows the same control plane can extend to heterogeneous compute.

The current experiment compares an exact classical solver with QAOA on the same allocation problem - without claiming quantum advantage.

A.AI

OverviewTasksQuantum Lab

Bob integration configure

RESEARCH EXTENSION · HETEROGENEOUS COMPUTE EXPERIMENT

Quantum Lab

Compare exact classical enumeration against an optional Qiskit QAOA experiment on the same batch allocation problem. The exact solver is the V1 reference. QAOA never bypasses Tessera's hard constraints.

Why this matters:

Tessera's long-term control plane can allocate not only among agent workflows, but among different computational backends while maintaining the same policy and verification boundary. This is a research experiment — it does not claim quantum advantage.

BATCH ALLOCATION PROBLEM

3 tasks · 6 route candidates

Compare solvers

TASKS & CANDIDATES

docs

FAST

cost 8 · latency 6 · failure penalty 1

ASSURANCE

cost 30 · latency 25 · failure penalty 0.5

billing

INVESTIGATE

cost 25 · latency 18 · failure penalty 8

ASSURANCE

cost 40 · latency 28 · failure penalty 1

OBJECTIVE & BUDGET

Shared budget

100 units

Cost weight

0.4

Latency weight

0.2

Failure penalty weight

0.4

The solver minimizes the weighted sum of cost, latency, and failure penalty across all assigned workflows, subject to the shared budget constraint. Every candidate assignment produced by either solver is independently re-validated by Tessera.

Qiskit

Research extension

Exact classical

0.26 ms

Qiskit QAOA

~6.36 s

Same assignment + objective

Tessera validates the same hard constraints regardless of backend.

NO QUANTUM ADVANTAGE CLAIM

Builder credibility

Open-source Qiskit contributor: investigated issue #16688 and opened PR #16944 on empty loose-basis transpilation behavior, including regression coverage and maintainer-requested refinements.

Long-term Tessera: agents + models + tools + CPU/GPU + classical/hybrid/quantum

Three enterprise use cases for outcome-aware orchestration.

Same primitive, different operating environments: route the work, verify the result, preserve evidence, then improve the next decision.



USE CASE
1

Enterprise AI software maintenance

Platform engineering + coding-agent fleets

Route routine fixes through lightweight workflows; escalate sensitive or failure-prone changes to deeper assurance.

VALUE Lower unnecessary agent effort, faster routine work, less rework.



USE CASE
2

Regulated change control

Finance · healthcare · critical infrastructure

Policy-gate agent workflows, independently verify changes, and retain immutable evidence for later review.

VALUE Safer automation, stronger auditability, clearer accountability.



USE CASE
3

Multi-agent + compute orchestration

Models · tools · CPU/GPU · classical/quantum

Allocate work across specialized backends while keeping one policy, evidence, and verification boundary.

VALUE Optimize cost, latency, and reliability without blindly trusting the backend.

COMMON THESIS Lower unnecessary workflow cost · Less rework · More independently verified successful outcomes

Potential use cases - enterprise ROI not yet validated at scale.